Programación Web

Taller de Programación

UNIVERSIDAD DE LOS LAGOS

CONTENIDOS

- ¿Qué es Git?
- Características
- ¿Cómo instalar Git?
- Estados de un archivo
- ¿Cómo trabajar con Git?

INTRODUCCIÓN

Al trabajar con proyectos es muy común crear copias del proyecto que van conteniendo las versiones de nuestros datos, esto puede ser complejo para una persona, pero es mucho mas complejo si se trata de un equipo de personas. En esta sesión se explicará que es un sistema de VERSIONAMIENTOS y cómo generar un REPOSITORIO que nos permitirá compartir nuestro proyecto en la nube y poder trabajar en equipo. En esta oportunidad se entregará el principio básico para crear un repositorio, en futuras secciones avanzaremos al trabajo en la nube y trabajo colaborativo en la nube mediante Github.

¿QUE ES GIT?

- GIT es un software de control de versiones diseñado por Linus Torvalds, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente. Su propósito es llevar registro de los cambios en archivos de computadora incluyendo coordinar el trabajo que varias personas realizan sobre archivos compartidos en un repositorio de código.

CARACTERÍSTICAS

- De esta herramienta se pueden menciona las siguientes características:

- No depende de un repositorio central. **El repositorio se crea en NUESTRO computador, para luego subir una copia a un repositorio en la nube**.

- Es software libre.

- Con ella podemos mantener un historial completo de versiones

- Podemos movernos, como si tuviéramos un puntero en el tiempo, por todas las revisiones de código y desplazarnos una manera muy ágil.

- Es muy rápida.

CARACTERÍSTICAS

- Tiene un sistema de trabajo con ramas que lo hace especialmente potente.

- En cuanto a la funcionalidad de las ramas, las mismas están destinadas a provocar proyectos divergentes de un proyecto principal, para hacer experimentos o para probar nuevas funcionalidades.

- Las ramas pueden tener una línea de progreso diferente de la rama principal donde está el core de nuestro desarrollo. En algún momento podemos llegar a probar algunas de esas mejoras o cambios en el código y hacer una fusión a nuestro proyecto principal, ya que todo esto lo maneja Git de una forma muy eficiente.

• ¿CÓMO INSTALAR GIT?

- **Instalación en Windows**

- También hay varias maneras de instalar Git en Windows. La forma más oficial está disponible para ser descargada en el sitio web de Git. Descargar Git desde el siguiente enlace: https://git-scm.com/downloads

Descargar y luego ejecutar para instalar Git.

• ESTADOS DE UN FICHERO GIT

- Existen tres tipos de estado de un fichero Git:

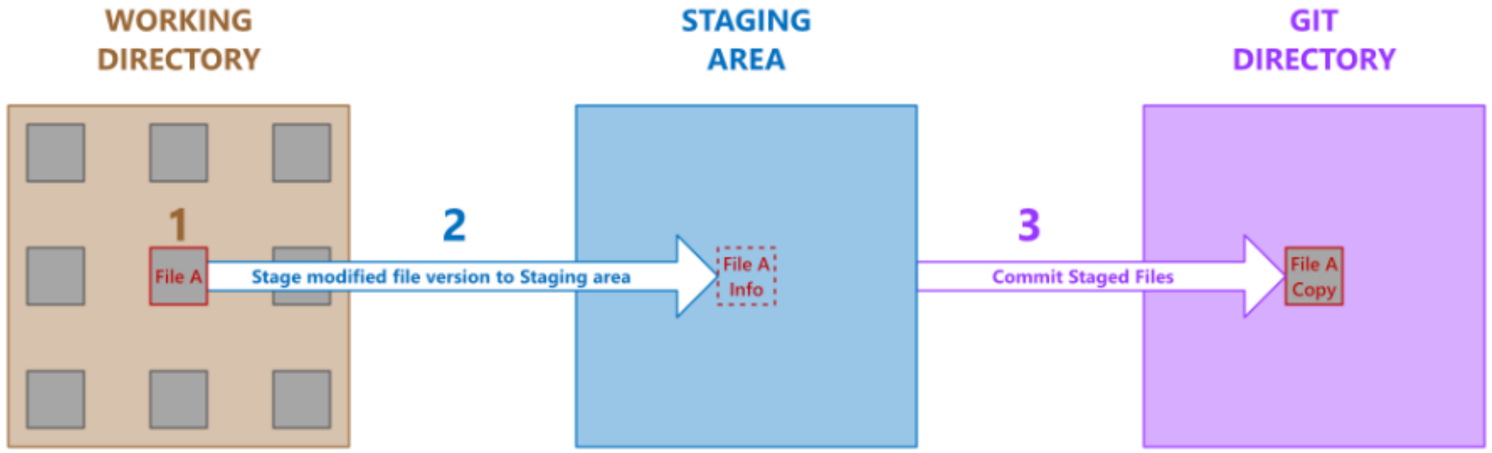
- **Confirmado**: Tenemos un fichero con el que hemos estado trabajando, hemos aprobado todos sus cambios y va en una nueva revisión, es decir, una especie de paquete dónde van todos los cambios.

- **Modificado**: Estamos trabajando en el directorio de trabajo (o working directory) y consideramos que ese cambio debe de ir en una revisión, en esa especie de paquete, para formar la revisión.

- **Preparado**: Hemos marcado un archivo para que vaya una revisión Git (repositorio)

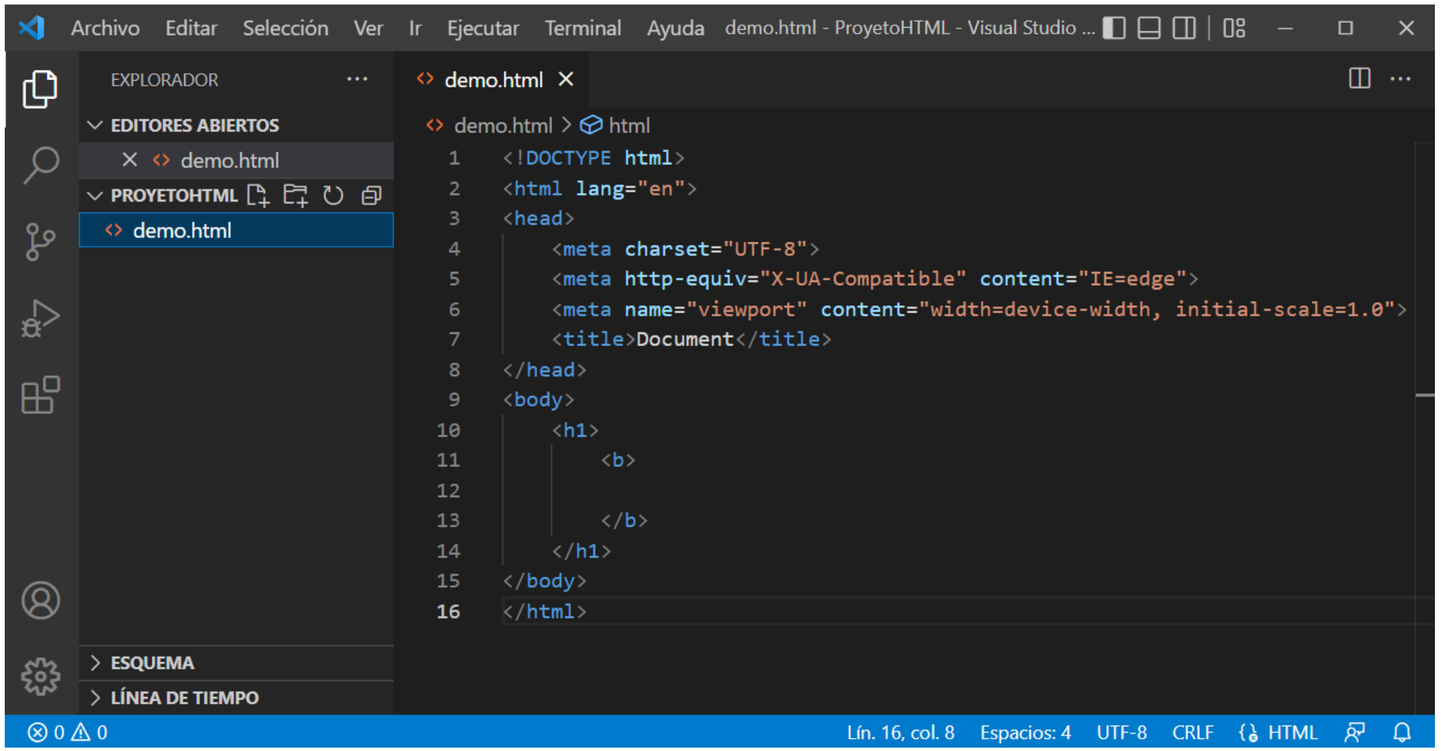
• ESTADOS DE UN ARCHIVO GIT

- Al crear un archivo en la carpeta está en el paso 1 Working Directory. Luego con un comando de Git se pasa ese archivo, o varios, al área de Staging. Finalmente, con otro comando de Git, se pasa el archivo al área Git Directory. Esta última área corresponde a los archivos del proyecto actualizados y listos para compartir.



• ¿CÓMO TRABAJAR CON GIT?

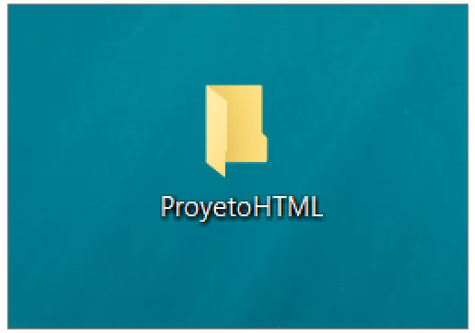
- Para la demostración utilizaremos el “ProyectoHTML” creado con anterioridad, pero a nivel de archivos, desde el escritorio.

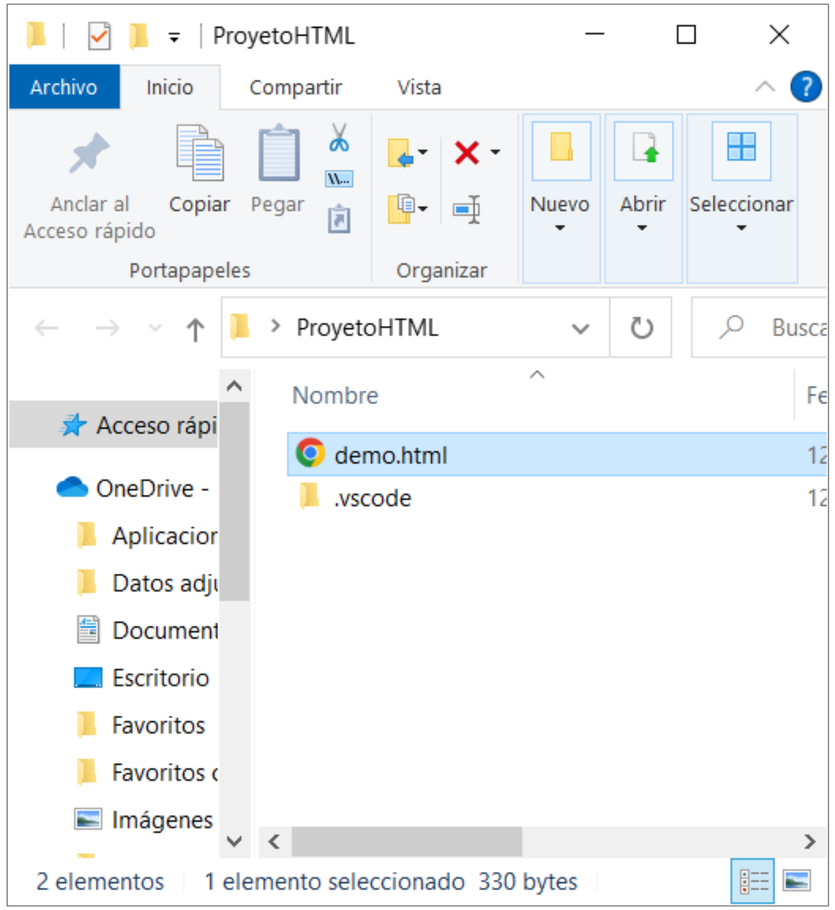


Proyecto a utilizar

• ¿CÓMO TRABAJAR CON GIT?

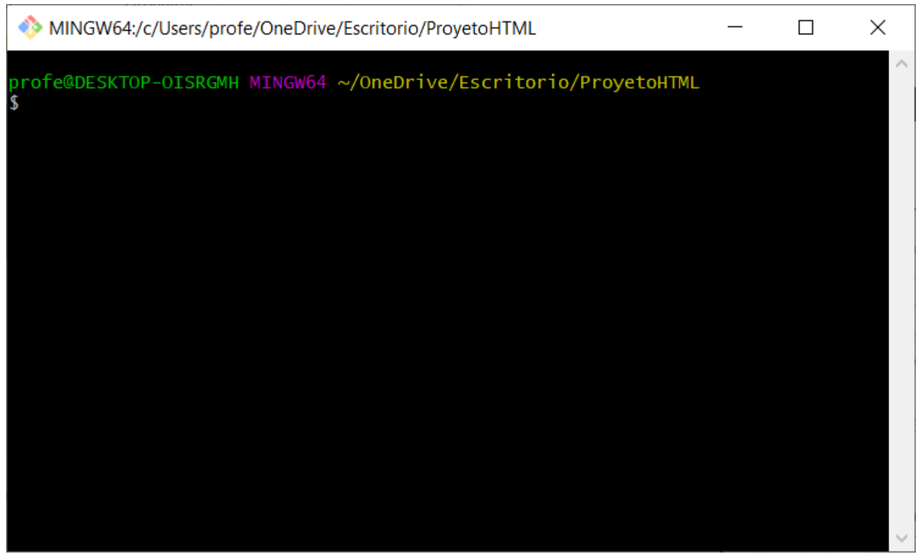
- Paso n°1: Ingresar a la carpeta del proyecto.

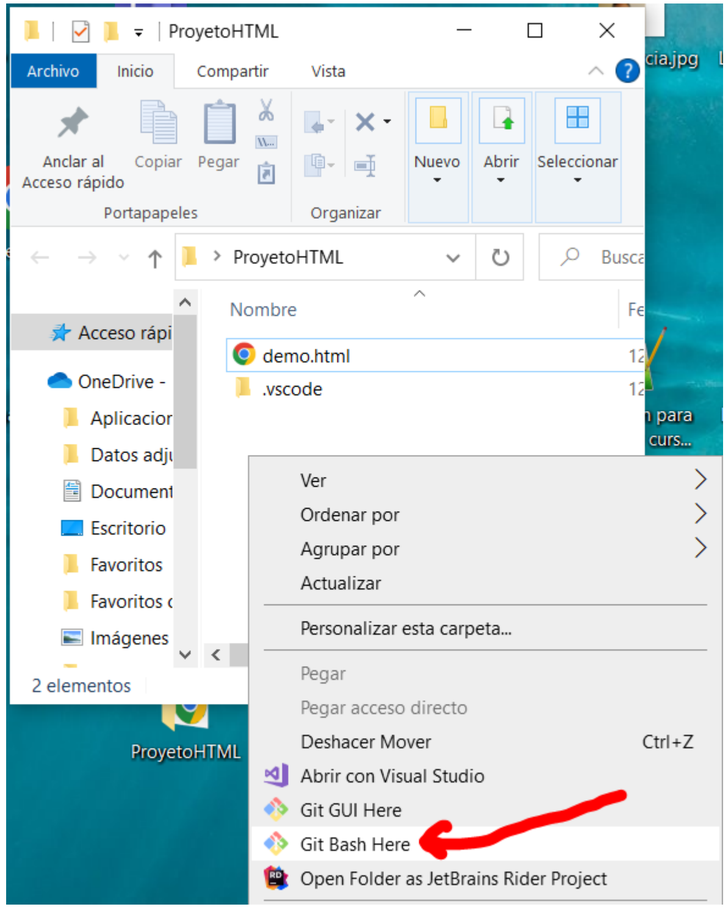




• ¿CÓMO TRABAJAR CON GIT?

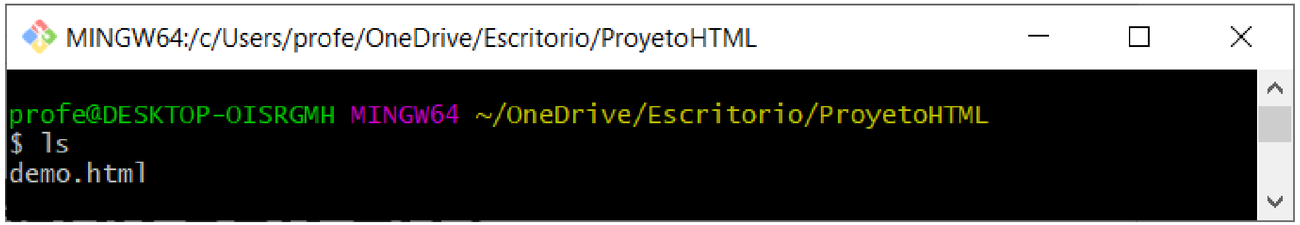
- **Paso n°2**: Ubicar el puntero del mouse en un área en blanco (debajo de .vscode) y presionar el botón derecho del mouse. Luego, en el menú contextual, seleccionar **Git Bash Here**. Esta acción abrirá una consola para escribir comandos y trabajar con el repositorio para este proyecto.





• ¿CÓMO TRABAJAR CON GIT?

- **Paso n°3**: Frente al prompt ($), de Git Bash, escribir el comando LS para mostrar los archivos de la carpeta, en este momento solo uno (demo.html).

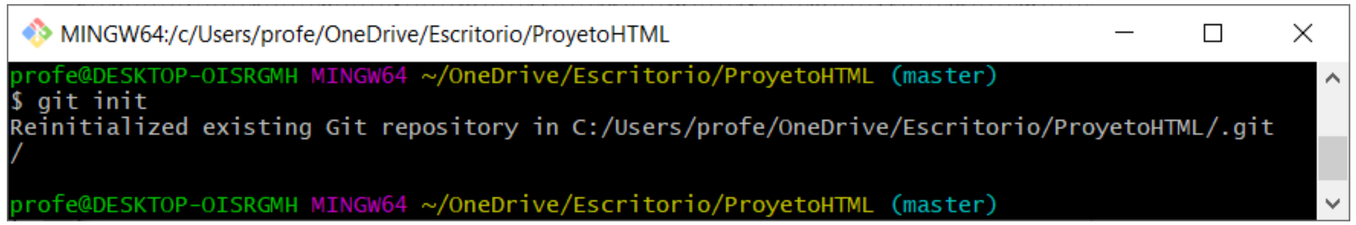


- **Paso n°4**: Realizar configuraciones de parámetros globales:
- $ git config --global user.name Francisco Calfún
- $ git config --global user.email francisco.calfun@ulagos.cl
- $ git config --global --list

- El signo $ no se escribe, es el prompt.
- Debe remplazar los datos personales por su nombre y correo.
- La última línea muestra que las configuraciones quedaron grabadas.

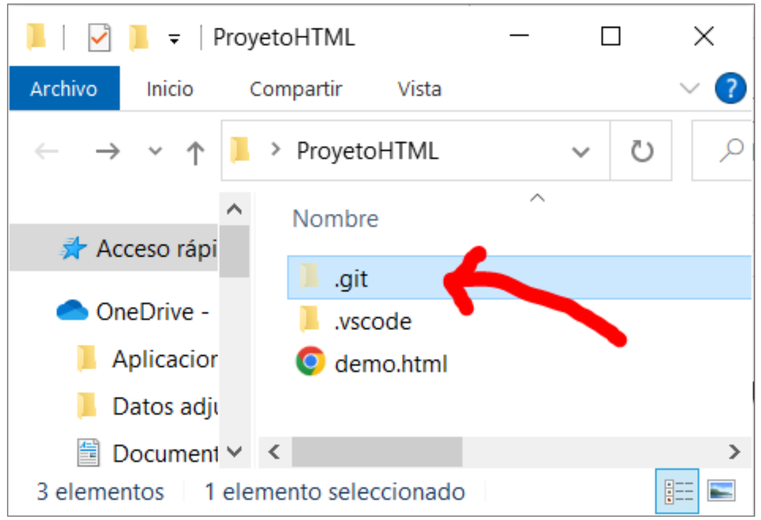
• ¿CÓMO TRABAJAR CON GIT?

- **Paso n°5**: Ahora ingresamos el comando “git init” para crear un REPOSITORIO para esta carpeta (proyecto).



- Esto genera una carpeta oculta llamada .git donde se almacena el repositorio, donde “repositorio” es una carpeta con los archivos del proyecto que están actualizados y “agregados” al repositorio.

•

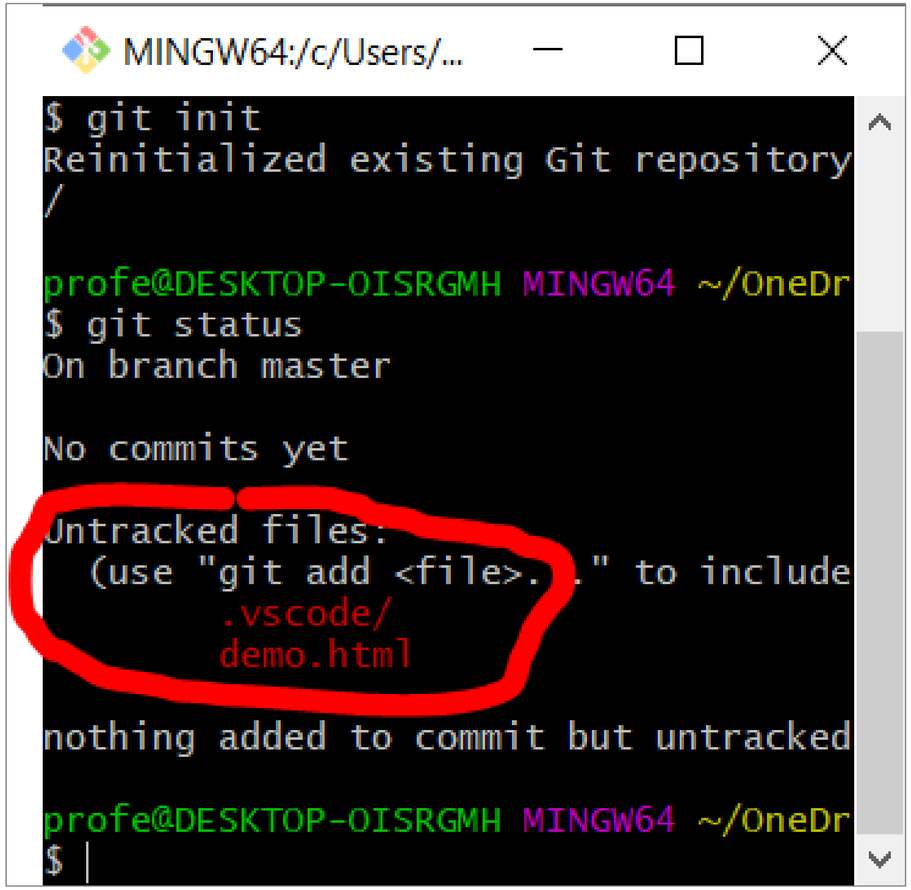


• ¿CÓMO TRABAJAR CON GIT?

- **Paso n°6:** Para ver los archivos actualizados, pero “sin seguimiento” (sin enviar al repositorio) utilizaremos el comando “git status”.

- La respuesta del comando “status” muestra que hay dos elementos Untracked (sin seguimiento), estos son: .vscode/ y demo.html

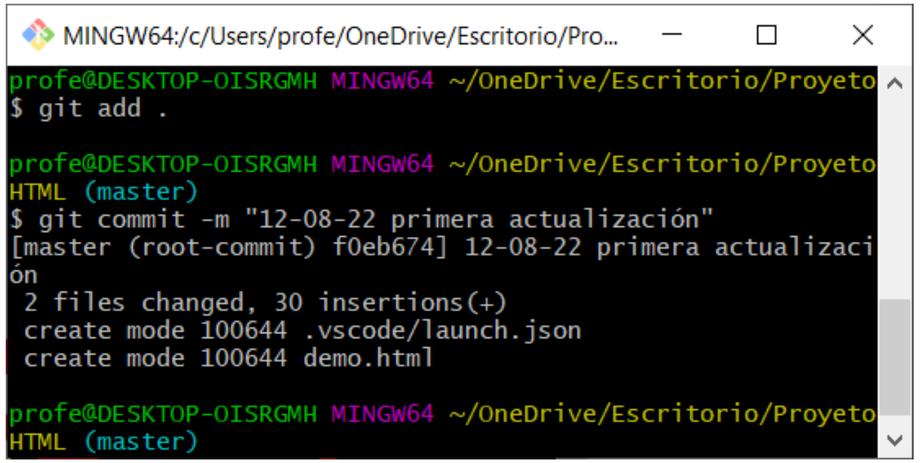
-



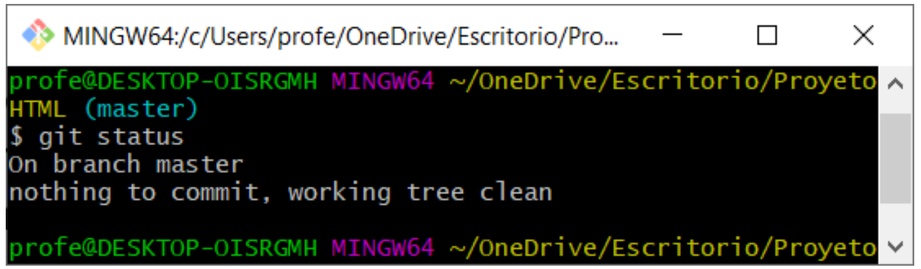
• ¿CÓMO TRABAJAR CON GIT?

- **Paso n°7:** Para pasar los archivos al repositorio usaremos el comando “add” seguido de un “commit”.

- $ git add .
- $ git commit -m “01-09-25 primera actualización“
- $ git status



- El comando add los “prepara” para ser enviados al repositorio. Es el commit quien envía todos los archivos “preparados” al repositorio, con –m podemos agregar un comentario sobre qué tipos de actualizaciones se realizaron al los archivos o proyecto.
- Aplicamos git status para demostrar que ya no hay archivos sin seguimiento (Untracked).



¿CÓMO TRABAJAR CON GIT?

- **Paso n°8**: En caso de agregar mas archivos al proyecto o actualizar los ya existentes hay que repetir el paso n°7.

- De esta manera estamos trabajando con "**VERSIONAMIENTO**" en nuestro propio computador. Las diferente versiones de nuestro proyecto corresponde a los **COMMIT** que vamos realizando.

- Pronto aprenderemos cómo volver a una versión anterior y cómo compartir mi repositorio en la nube con GITHUB.